

# Arabic Learning Game

JIB Team 4126

Brandon Harris, Daniel Park, Ivan Shi, Michael Tang, Ryan Yiu

Client: Dr. Natalie Khazaal

Repository: <https://github.com/Bman3141/JIB-4126-Learn-Arabic-Game>

# Table of Contents

|                                  |    |
|----------------------------------|----|
| Table of Contents .....          | 2  |
| Table of Figures .....           | 3  |
| Terminology .....                | 4  |
| Introduction .....               | 5  |
| Background.....                  | 5  |
| Document Summary .....           | 5  |
| System Architecture .....        | 6  |
| Introduction.....                | 6  |
| Rationale .....                  | 6  |
| Static System Architecture ..... | 7  |
| Dynamic System Architecture..... | 9  |
| Component Design.....            | 11 |
| Introduction.....                | 11 |
| Static Component Design .....    | 11 |
| Dynamic Component Design .....   | 12 |
| Data Design.....                 | 13 |
| Introduction.....                | 13 |
| Database Use .....               | 13 |
| File Use.....                    | 14 |
| Data Exchange.....               | 14 |
| UI Design .....                  | 15 |
| Introduction.....                | 15 |

## Table of Figures

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Figure 1.1: Static System Architecture .....</b> | <b>7</b>  |
| <b>Figure 1.2: Dynamic System Architecture.....</b> | <b>9</b>  |
| <b>Figure 2.1: Static Component Design.....</b>     | <b>12</b> |
| <b>Figure 2.2: Dynamic Component Design.....</b>    | <b>13</b> |
| <b>Figure 3.1 Data Design.....</b>                  | <b>15</b> |
| <b>Figure 4.1 Home Screen.....</b>                  | <b>15</b> |
| <b>Figure 4.2 Levels Screen.....</b>                | <b>15</b> |
| <b>Figure 4.3 Settings Screen.....</b>              | <b>15</b> |
| <b>Figure 4.4 Character Select Screen.....</b>      | <b>15</b> |
| <b>Figure 4.5 Grocery Map.....</b>                  | <b>16</b> |
| <b>Figure 4.6 Drag &amp; Drop Quiz Screen.....</b>  | <b>16</b> |
| <b>Figure 4.7 Book Layout.....</b>                  | <b>16</b> |
| <b>Figure 4.8 Dialogue Event.....</b>               | <b>16</b> |

# Terminology

**Asset pack:** A collection of digital assets, such as images, sounds, and scripts.

**AWS (Amazon Web Services):** A cloud computing platform provided by Amazon that offers services including computing power, storage, and databases.

**Back-end:** The server-side (non-interactable) part of a software application or website.

**CSS (Cascading Style Sheets):** A stylesheet language used to describe the presentation and layout of a web page written in HTML or XML.

**Front-end:** The client-side part of a software application or website.

**HTML (Hypertext Markup Language):** The standard markup language used to create and structure content on the web.

**JavaScript:** A programming language used for interactive content on web pages.

**LEMP (Linux, Nginx, MySQL, PHP) Stack:** A set of open-source software used together for web development: Linux, Nginx, MySQL, and PHP.

**Level:** In video games, a specific area or stage that players progress through.

**Linux:** An open-source operating system kernel.

**MySQL (Structured Query Language):** A relational database management system that uses SQL for querying and managing data.

**NGINX (Engine-X):** A web server and reverse proxy server used to handle web traffic.

**NPC (Non-player Character):** Characters in a game that are not controlled by players.

**PHP (Hypertext Preprocessor):** A server-side scripting language for web development.

**RPG (Role-playing Game):** A genre of games where players assume the roles of characters in a fictional setting and undertake adventures or quests.

**Sprite:** A 2D image or animation used in games and applications for characters or objects.

**Tiled:** A tool used for creating tile-based maps and levels for games.

**Tileset:** A collection of images used to construct visual elements of a game level or map.

**UI (User Interface):** The means by which users interact with a software application or system, including visual elements like buttons, menus, and forms.

**Webserver:** A server that hosts websites and serves web pages to users over the internet.

# Introduction

## Background

Our project is a game developed for intermediate Arabic language learners. The main objective is for users to interact with the engaging learning elements in a game format to improve their language retention and mastery. The game is developed in JavaScript, HTML, and CSS to run as a web application, hosted on a webserver running a LEMP stack. The software Tiled will be used to apply existing asset-packs to style the game environment and user interface. The webserver and database will be based on AWS. The game is playable on both mobile devices and personal computers as it is hosted online. Our game will be structured as an RPG revolving around an Arabic character, who the player will be able to control and interact with Arabic language puzzles to progress the character's story. Users will be able to make user accounts and play one level on release, as well as save their progress to the cloud. An administrator can check each user's progress.

## Document Summary

The System Architecture section provides a high-level view of our system, with our LEMP server stack from AWS for user accounts and game hosting interacting with the large module of JavaScript/HTML/CSS for the game.

The Component Design section provides a more detailed view of each component's function for the game. Specific classes/modules that drive the JavaScript for character movement, game environment, puzzles, and user interface are outlined. The interworking of technologies in the LEMP server stack is also described.

The Data Storage Design section provides a view of our MySQL database setup for storing user and administrator account information, as well as game state storage for each user.

The UI Design section shows the main user interface in the beginning menus, the movement controls for the player in a game level, and the more specific interactable puzzle designs in the game level.

# System Architecture

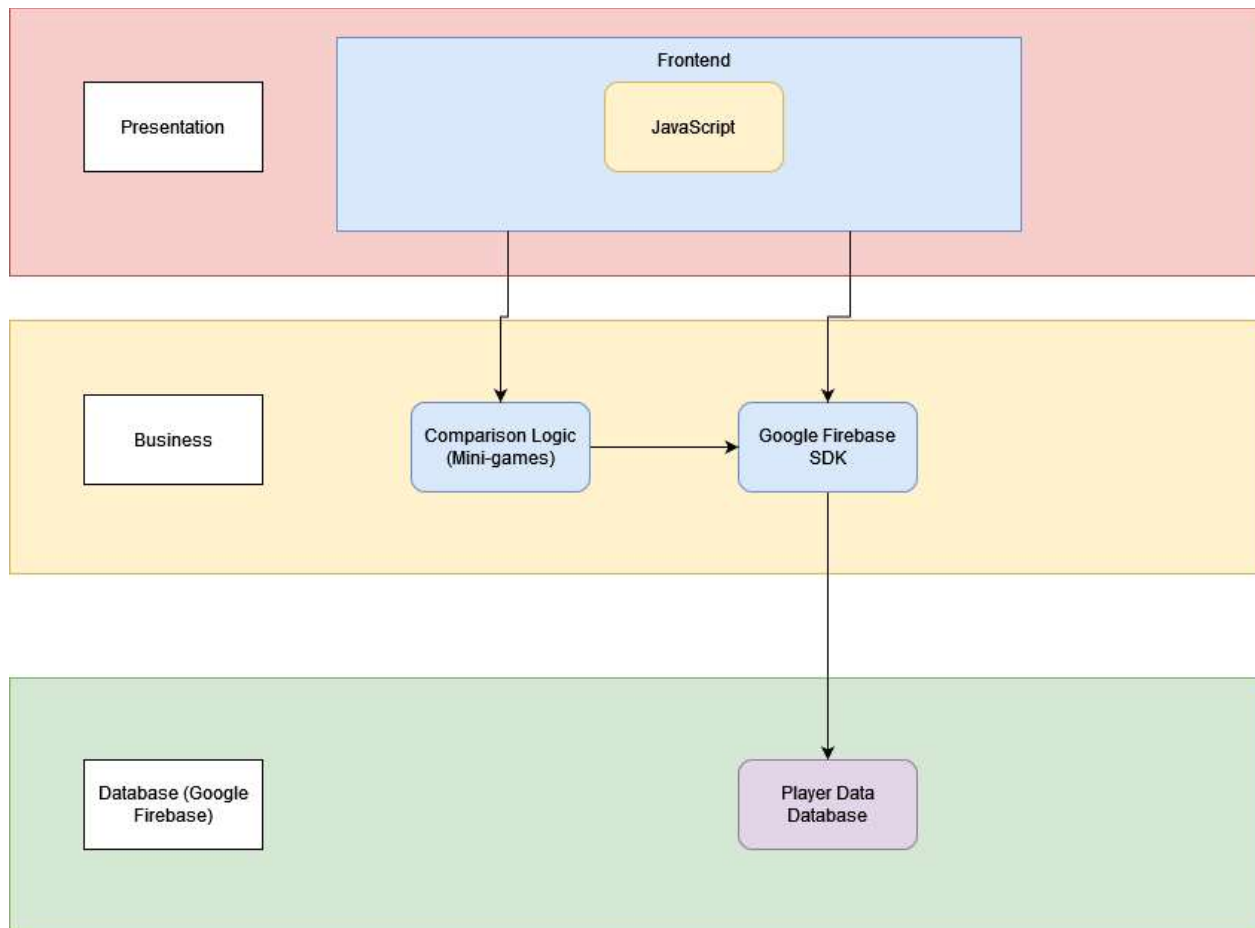
## Introduction

To better understand the structure of the game, the static and dynamic architectural designs provide visuals for the overall system design. Our overall application is a web-based RPG game that will have a client-server architecture, so that the game is loaded client side and requests for login and progress are sent to server side. This relationship is outlined with the components in the static system architecture. The dynamic system architecture describes the more specific runtime behavior of our system, showing how the components will function in common usage scenarios.

## Rationale

The bulk of our work will focus on the inner components of the game itself, with few requests to the server/database aside from login and progress saving. As the application will not have an auto saving feature implemented, this greatly reduces the number of database queries. This prompted the use of Firebase with its serverless architecture, reducing the complexity of managing a full, traditional database as we do not expect to handle excessive queries. The choice of JavaScript for the client side was supported by the need for users to be able to access the game on both mobile and desktop devices, which is possible through browsers running JavaScript. One important note is that our architecture does not describe the inner components of the game itself, which is almost fully JavaScript.

## Static System Architecture



**Figure 1.1: Static System Architecture**

### Description

Our application is comprised of three major components including the Presentation, Business, and Database layers. These components work together to create an engaging and educational experience for users, with language challenges integrated into the game environment.

The **Presentation** is where users interact with the game environment on the client side. Built off primarily JavaScript, it allows players to navigate the game's story, interact with Non-Playable Characters (NPCs), and engage with various game elements. The interface is designed to be intuitive, supporting desktop, mobile, and web browsers. In addition to character navigation, the UI also includes dialogue systems and dynamic elements that allow the user to interact with Arabic resources and prompts that trigger the language puzzles.

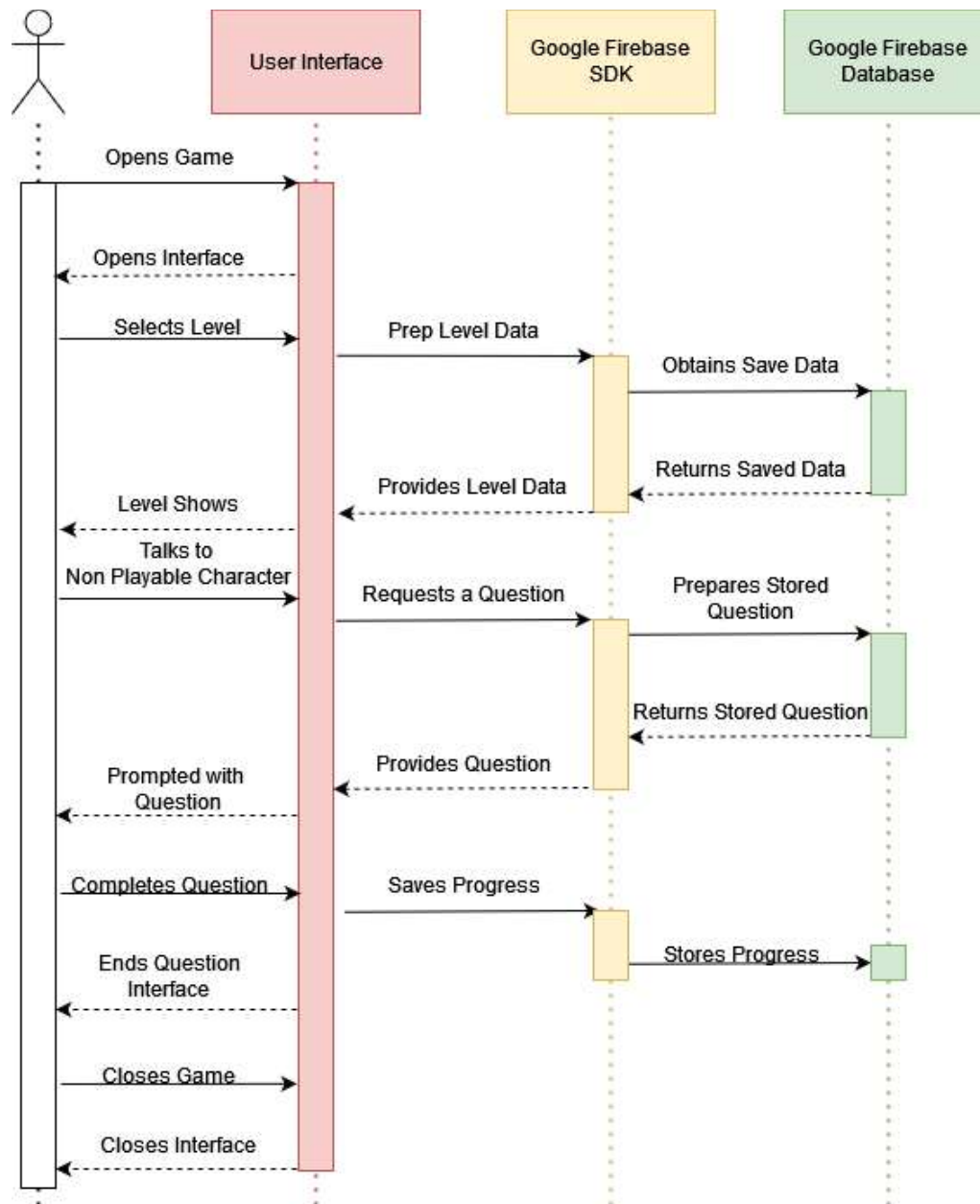
The **Business** layer handles the backend of the game; namely, the logic behind the interactions that the user will have with the language puzzles, as well as the overall story progress/completion. Included in this learning experience are NPCs. Each NPC is tied to specific language puzzles, triggered by player interactions. The NPC interaction logic is designed to ensure smooth dialogue exchanges and trigger appropriate puzzles based on the storyline. As the player progresses, the difficulty of the puzzles will gradually increase to align with the growth of the user's knowledge.

Our game relies on a **Database**, using a cloud-based backend built with Google Firebase to store player profiles, language learning progress, and game state. The Data Management System facilitates real-time synchronization between the player's device and the cloud, allowing users to pick up where they left off across multiple devices. Additionally, it stores information such as dialogue history, puzzles, and user achievements, enabling dynamic updates without disrupting gameplay.

The game supports multiple input methods to enhance accessibility, including keyboard, mouse, and touch controls. As the game is based on the web, it allows for further accessibility across a variety of devices, which aligns with the notion that our intended player base is comprised of primarily Georgia Tech Arabic students. The input system allows players to navigate the environment, select dialogue options, and interact with language puzzles seamlessly.



## Dynamic System Architecture



**Figure 1.2: Dynamic System Architecture**

### Description

The dynamic architecture design diagram depicts the control flow within the system and illustrates the interactions between key game components as the exchanging of data occurs. The presented scenario outlines the runtime behavior when a user successfully completes a quiz.

Upon start up, the **UserInterface** component serves as the main interface layer between the user and the game application. In the scenario, the **UserInterface** handles multiple user requests, ranging from game settings, level selection and the completion of in-game tasks.

Beneath the interface layer, the **Google Firebase SDK** component serves as a loading dock and prepares the **Google Firebase Database** for saving each user's game data. Upon user input, **Google Firebase SDK** fetches their progress in the first level and the number of quizzes they have completed from **Google Firebase Database**. Upon successful data retrieval, **Google Firebase SDK** updates the current game state in **UserInterface**.

The **Google Firebase Database** component is responsible for storing each user's game progress upon completing various mini-milestones presented in the game, such as obtaining a story-specific item, key, or successfully completing a quiz. Upon successful retrieval of data, the **Google Firebase Database** component is called to manage and maintain user data in the current game state. This mainly involves constantly storing user specific data, such as game settings, current level, obtained items, completed quizzes, and degree of completion in the current level. In the scenario demonstrated above, the **Google Firebase Database** preserves progress in the current level, a list of questions for in-game quizzes, and the number of completed quizzes. The repeated updates of user data to the database allow each session to be saved after use and each user to continue playing the game from their previous session.

# Component Design

## Introduction

In Figures 2.1 and 2.2, we have depicted the static and dynamic components of our system architecture. The static structure is represented using a class diagram, which highlights the direct relationships and connections between various classes and components. The dynamic aspects are illustrated through an interaction overview diagram, which outlines the process of the user saving their progress before exiting the application, showcasing the interaction with the database.

In these diagrams, our front-end game layer is shown in red. Yellow represents our business layer, which bridges front-end and back-end. The database, represented in green, has not yet been implemented.

## Static Component Design

In Figure 2.1 we have the static detailed components of our system (for now just the red front-end game). The Main [Menu], Customize, Levels, and Settings are all front-end screens presented to the user, with a few buttons with navigational properties. Settings relies on Audio to adjust its volume sliders. In Game which is the actual game engine, we rely on Init to initialize the Overworld. Overworld will then set up the canvas to load graphics onto, as well as the OverworldMap and DirectionInput. DirectionInput handles the user keyboard inputs to the game. OverworldMap handles loading the game map as well as multiple game objects on the map, so GameObject is a part of it. GameObject is associated with Sprite, our character and NPC models. Person is an extension of GameObject, and it controls the player Sprite specifically. The Business and Database layers are not present at this point of development.

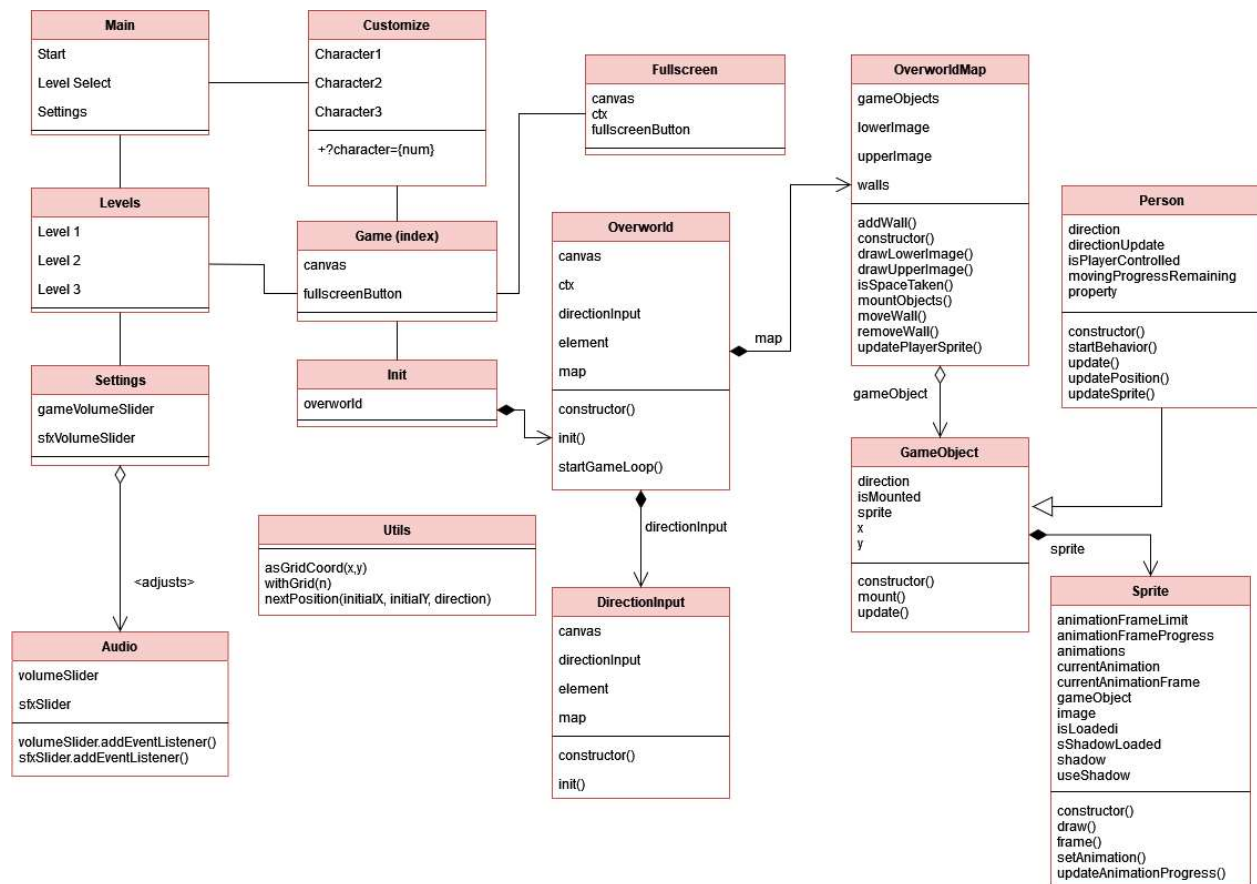
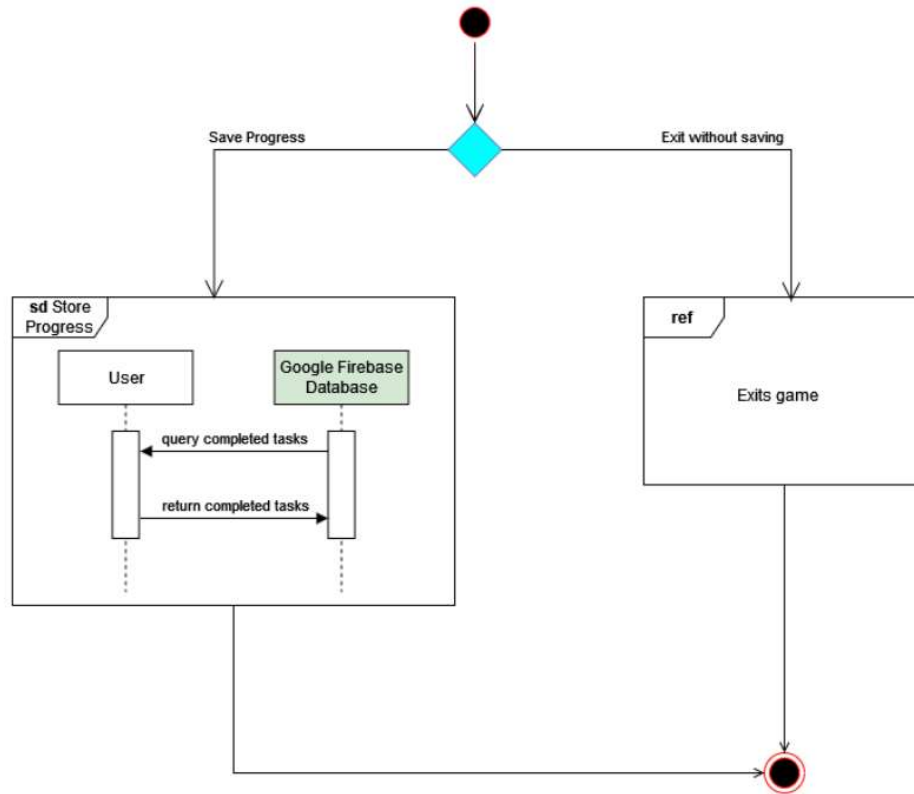


Figure 2.1: Static Component Design Diagram

## Dynamic Component Design

In Figure 2.2 we depict an Interaction Overview Diagram of a workflow that would occur while interacting with the game. The model demonstrates the process of a user choosing to save a game before exiting the application. If the user decides to close the game without saving, the program would simply close, and the user's saved data would be lost. However, if the user decides to save the data, the data is queried by the **Google Firebase Database** and once it is received, it is saved.



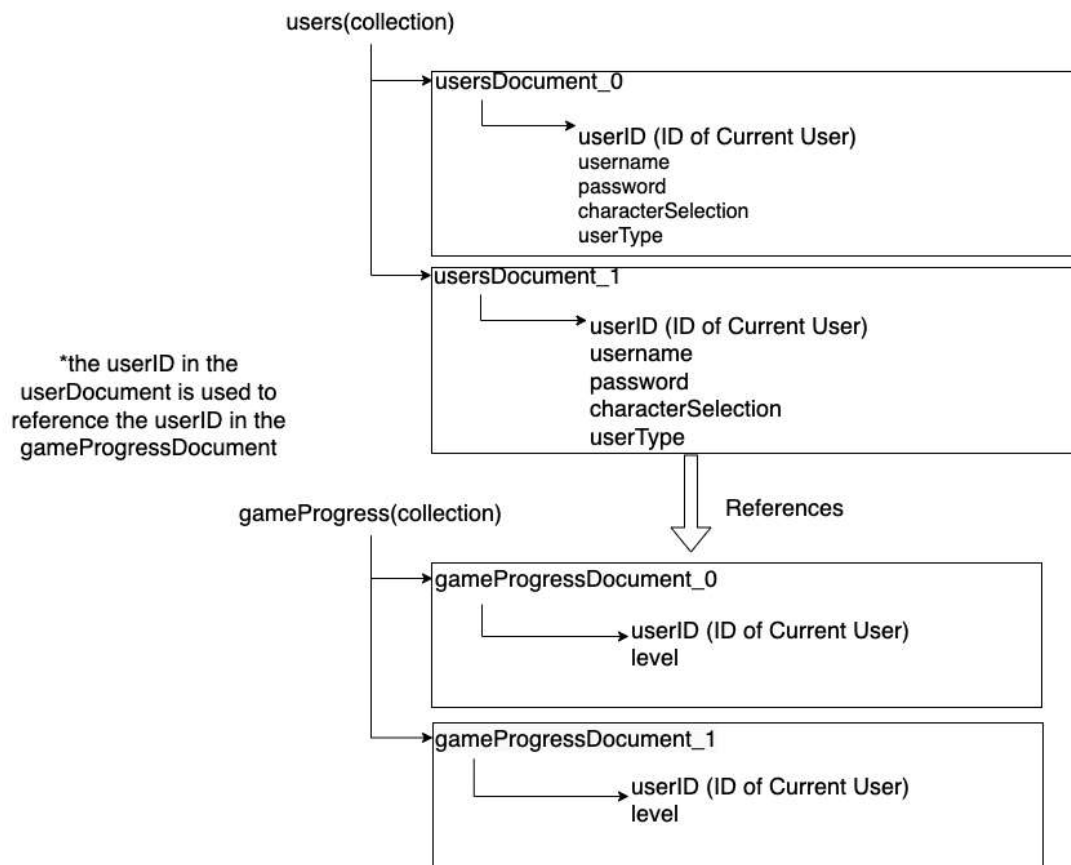
**Figure 2.2: Dynamic Component Design**

## Data Design

### Introduction

The diagram below shows how we use Google Firebase to authenticate user login and track game progress. A document-database diagram was the best option for our project, as Google Firebase saves data in the form of collections with documents within those collections. In terms of format, Google Firebase stores data in the form of JSON objects. The only data that we will need to store are username, password, character selection (saves sprite selection), user type (admin or student), and game level (level 1, level 2, ...). After the diagram, the File Use and Data Exchange will be discussed.

### Database Use



**Figure 3.1 Data Design**

## File Use

There is no file usage used for data storage.

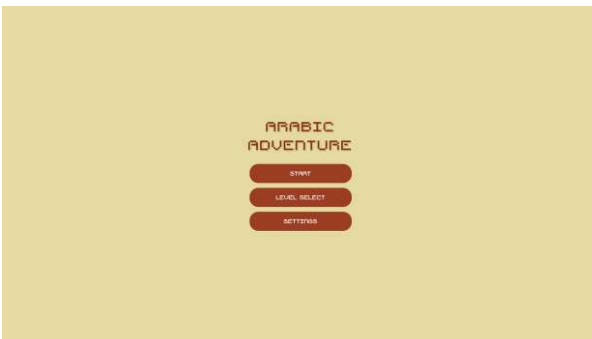
## Data Exchange

Our application uses Google Firebase to store user login information and in game progress between their browsers and the server. User authentication will work by parsing from the username and password fields in the application. They will need to be stored as JSON (JavaScript Object Notation) Objects, where we will utilize Google Firebase Authentication SDK to validate user information. For the actual game data that will be passed between the client browser and the server, JSON will be used to format the data.

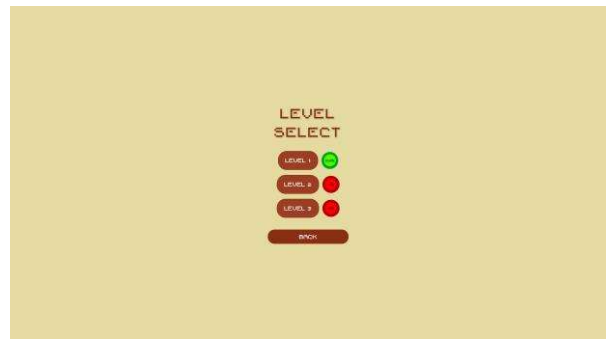
# UI Design

## Introduction

In this section we present the User Interface (UI) of our web-based game, where users can interact with the screens shown below to enter the game and interact with game elements. We discuss our UI design decisions and the purpose of all the main screens the user will interact with. Overall, our UI style was based off the popular pixel-art RPG genre for the story of Olivia. Most buttons and interactive screens were made minimal and simple to use (which makes interaction easier for mobile users as well).



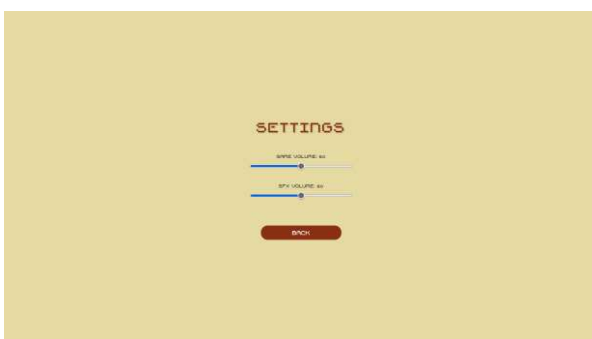
**Figure 4.1 Home Screen**



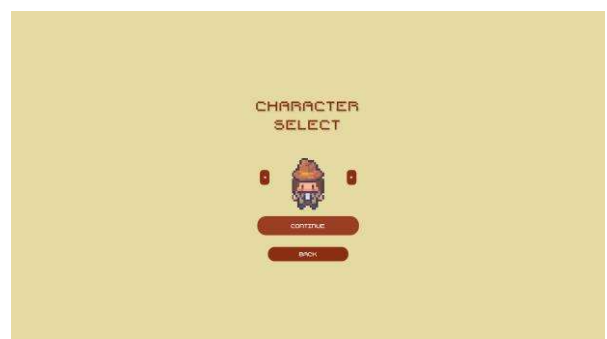
**Figure 4.2 Levels Screen**

Figure 4.1: When the user opens the “Arabic Adventure” game, they are greeted with an intuitive main menu screen designed to facilitate easy navigation.

Figure 4.2: When the user navigates to the Level Select screen, they are presented with a visually engaging and user-friendly interface that allows them to choose from three game levels. This screen is designed with a straightforward approach because users need to be able to easily select their desired level and understand their progress.



**Figure 4.3 Settings Screen**



**Figure 4.4 Character Select Screen**

---

Figure 4.3: The user can adjust volume preferences for the game window (background music and sound effects) to improve game immersion.

Figure 4.4: After the user enters the level screen or presses start, they are greeted with the character selection where they can customize the character's hat color.



**Figure 4.5 Grocery Map**



**Figure 4.6 Drag & Drop Quiz Screen**

---

Figure 4.5: The Grocery Store map is one of the main priorities of our client, where the user can engage in interactive events and dialogue to practice and improve their Arabic comprehension and cultural knowledge.

Figure 4.6: One of the core “language challenges” of the game is the interactive draggable element quiz, where users can complete Arabic translation puzzles to test their knowledge.



**Figure 4.7 Book Layout**



**Figure 4.8 Dialogue Event**

---

Figure 4.7: The user can access an Arabic Book with information that may assist the user with their progression of the game.

Figure 4.8: The user can interact with certain NPCs to engage in dialogue in either English or Arabic to receive further conversational practice and comprehension.



